

# Binary Trees

---

**PALAKURTHI LALITH PRAKASH**

**2303A52012**

**31.08.2026**

**1. Print the boundary nodes of a binary tree.**

```
class Solution {

    public static void leftc(Node root, ArrayList<Integer> res) {
        if (root == null) return;
        if (root.left == null && root.right == null) return;
        res.add(root.data);
        if (root.left != null) leftc(root.left, res);
        else leftc(root.right, res);
    }

    public static void rightc(Node root, ArrayList<Integer> res) {
        if (root == null) return;
        if (root.left == null && root.right == null) return;
        if (root.right != null) rightc(root.right, res);
        else rightc(root.left, res);
        res.add(root.data);
    }

    public static void leafc(Node root, ArrayList<Integer> res) {
        if (root == null) return;

        if (root.left == null && root.right == null) {
            res.add(root.data);
            return;
        }

        leafc(root.left, res);
        leafc(root.right, res);
    }

    ArrayList<Integer> boundaryTraversal(Node root) {
        ArrayList<Integer> res = new ArrayList<>();

        if (root == null) return res;
        if (root.left == null && root.right == null){
```

```

        res.add(root.data);
        return res;
    }
    res.add(root.data);

    leftc(root.left, res);
    leafc(root.left, res);
    leafc(root.right, res);
    rightc(root.right, res);

    return res;
}
}

```

## 2. Print the left view of a binary tree.

```

/*
class Node {
    int data;
    Node left, right;

    Node(int val) {
        this.data = val;
        this.left = null;
        this.right = null;
    }
}
*/

class Solution {
    public ArrayList<Integer> leftView(Node root) {
        ArrayList<ArrayList<Integer>> ans= new ArrayList<>();

        Queue<Node> q= new LinkedList<>();
        q.add(root);
        while(!q.isEmpty()){
            int n= q.size();
            ArrayList<Integer> al= new ArrayList<>();
            for(int i=0;i<n;i++){
                Node cur=q.poll();
                if(cur!=null){
                    if(cur.left!=null) q.add(cur.left);
                    if(cur.right!=null) q.add(cur.right);
                    al.add(cur.data);
                }
            }
        }
    }
}

```

```

        ans.add(al);
    }
    ArrayList<Integer> a= new ArrayList<>();
    for(ArrayList<Integer> al : ans){
        a.add(al.get(0));
    }
    return a;
}
}

```

### 3. Print the right view of a binary tree.

```

/*
class Node {
    int data;
    Node left, right;

    Node(int val) {
        this.data = val;
        this.left = null;
        this.right = null;
    }
}
*/

class Solution {
    public ArrayList<Integer> rightView(Node root) {
        ArrayList<ArrayList<Integer>> ans= new ArrayList<>();

        Queue<Node> q= new LinkedList<>();
        q.add(root);
        while(!q.isEmpty()){
            int n= q.size();
            ArrayList<Integer> al= new ArrayList<>();
            for(int i=0;i<n;i++){
                Node cur=q.poll();
                if(cur!=null){
                    if(cur.left!=null) q.add(cur.left);
                    if(cur.right!=null) q.add(cur.right);
                    al.add(cur.data);
                }
            }
            ans.add(al);
        }
        ArrayList<Integer> a= new ArrayList<>();
        for(ArrayList<Integer> al : ans){

```

```

        a.add(al.get(al.size()-1));
    }
    return a;
}
}

```

#### 4. Find the lowest common ancestor (LCA) of two nodes in a binary tree.

```

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p,
TreeNode q) {
        HashMap<TreeNode, TreeNode> parent = new HashMap<>();
        parent.put(root, null);
        findParents(root, parent);
        HashSet<TreeNode> ancestors = new HashSet<>();
        while (p != null) {
            ancestors.add(p);
            p = parent.get(p);
        }
        while (q != null) {
            if (ancestors.contains(q)) return q;
            q = parent.get(q);
        }

        return null;
    }

    static void findParents(TreeNode root, HashMap<TreeNode, TreeNode>
parent) {

        if (root == null) return;

        if (root.left != null) {
            parent.put(root.left, root);
            findParents(root.left, parent);
        }
    }
}

```

```

    }

    if (root.right != null) {
        parent.put(root.right, root);
        findParents(root.right, parent);
    }
}
}

```

5. Find the path from the root to a given node in a binary tree.

```

class Solution {
    public List<String> binaryTreePaths(TreeNode root) {
        List<String> ans = new ArrayList<>();
        if (root == null) return ans;
        dfs(root, "", ans);
        return ans;
    }

    public void dfs(TreeNode node, String path, List<String> ans) {
        if (node == null) return;
        path += node.val;
        if (node.left == null && node.right == null) {
            ans.add(path);
            return;
        }
        path += "->";

        dfs(node.left, path, ans);
        dfs(node.right, path, ans);
    }
}

```

6. Construct a binary tree from inorder and preorder traversal.

```

class Solution {
    int preIndex = 0;
    public TreeNode build(int[] preorder, int[] inorder,
        Map<Integer, Integer> map, int start, int end) {
        if (start > end) return null;
        int rootVal = preorder[preIndex++];
        TreeNode root = new TreeNode(rootVal);
        int idx = map.get(rootVal);
        root.left = build(preorder, inorder, map, start, idx - 1);
        root.right = build(preorder, inorder, map, idx + 1, end);
        return root;
    }
}

```

```
}  
public TreeNode buildTree(int[] preorder, int[] inorder) {  
  
    Map<Integer, Integer> map = new HashMap<>();  
    int n = inorder.length;  
    for (int i = 0; i < n; i++) map.put(inorder[i], i);  
    return build(preorder, inorder, map, 0, n - 1);  
}  
}
```